

UNIVERSIDAD MESOAMERICANA

Ingeniería en Sistemas

Manual de Usuario

Compilador MicroC

Nombre:	Jose Angel Moreno Hernández
Carné:	202425514
Curso:	Autómatas y Lenguajes
Proyecto:	Compilador MicroC

2026

Contenido

1. Introducción
2. Requisitos del Sistema
3. Instalación y Ejecución
4. Descripción de la Interfaz
5. Funcionalidades Implementadas
6. Tokens Reconocidos
7. Validaciones Realizadas
8. Ejemplos de Uso
9. Recomendaciones de Uso
10. Conclusión

1. Introducción

El presente manual de usuario tiene como objetivo explicar el funcionamiento básico del sistema Compilador MicroC, una aplicación de escritorio desarrollada en Avalonia .NET para realizar análisis léxico de código fuente similar al lenguaje C.

El sistema permite escribir, abrir, guardar y analizar código fuente. Al presionar el botón Compilar, el programa ejecuta el analizador léxico y muestra una lista de tokens con su respectiva línea, lexema, número de token y tipo.

2. Requisitos del Sistema

- Sistema operativo: Windows, Linux, probado en entorno Ubuntu.
- .NET 8 instalado para ejecución desde código fuente.
- Terminal o entorno de desarrollo para ejecutar comandos.
- No requiere instalación adicional para la versión compilada, siempre que se ejecute en el entorno compatible donde fue publicada.

3. Instalación y Ejecución

3.1 Ejecución desde código fuente

1. Abrir una terminal dentro de la carpeta del proyecto.
2. Compilar el proyecto con el comando:

```
dotnet build src/MicroC.csproj
```

3. Ejecutar el programa con el comando:

```
dotnet run --project src/MicroC.csproj
```

3.2 Ejecución de versión compilada Release

1. Ir a la carpeta:

```
bin/Release/net8.0/linux-x64/publish/
```

2. Ejecutar el archivo:

```
./MicroC
```

4. Descripción de la Interfaz

La interfaz principal del sistema se compone de una barra superior de botones, un editor de código fuente y una consola de salida para los resultados del análisis léxico.

4.1 Barra superior de botones

Botón	Descripción
Nuevo	Limpia el editor para iniciar un nuevo archivo.
Abrir	Permite seleccionar y abrir un archivo existente.
Guardar	Guarda el contenido actual del editor.
Editar	Activa el modo de edición del código fuente.
Compilar	Ejecuta el análisis léxico del código ingresado.
Ayuda	Abre la ayuda o documentación del compilador.
Salir	Cierra el programa. Si existen cambios sin guardar, solicita confirmación.

4.2 Editor de código fuente

Es el área donde el usuario escribe o pega el código fuente que desea analizar. El editor utiliza una apariencia oscura y fuente monoespaciada para facilitar la lectura del código.

4.3 Resultado del análisis léxico

Es la consola de salida ubicada en la parte inferior. En esta sección se muestra línea, lexema, token, tipo, total de tokens encontrados y total de errores léxicos.

5. Funcionalidades Implementadas

- Creación de nuevo archivo.
- Apertura de archivos existentes.
- Guardado de archivos.
- Edición de código fuente.
- Análisis léxico del código ingresado.
- Generación de lista de tokens.
- Reconocimiento de palabras reservadas.
- Reconocimiento de identificadores.
- Reconocimiento de números enteros.
- Reconocimiento de números reales.
- Reconocimiento de comentarios de línea.
- Reconocimiento de comentarios de bloque.
- Reconocimiento de operadores simples y compuestos.
- Detección de errores léxicos.
- Conteo total de tokens y errores.
- Confirmación al salir si existen cambios sin guardar.
- Botón de ayuda con documentación del compilador.

6. Tokens Reconocidos

Tipo de token	Ejemplos
Palabras reservadas	int, float, double, return, if, while, for, void
Identificadores	main, x, precio, suma_total, valor_2
Números enteros	0, 10, 25, 100
Números reales	10.50, 3.1416, 0.5, 100.00
Comentarios de línea	// variable inicial
Comentarios de bloque	/* Este es un comentario */
Operadores y símbolos	+, -, *, /, =, ==, !=, >=, <=, &&, , ++, --, :, (,), {, }

7. Validaciones Realizadas

- Código vacío.
- Números mal formados.
- Números reales sin dígitos después del punto.
- Números con más de un punto decimal.
- Comentarios de bloque sin cerrar.
- Símbolos no reconocidos.
- Clasificación correcta entre identificadores y palabras reservadas.
- Diferenciación entre números enteros y números reales.

Ejemplos de errores léxicos:

```
10.5.3
8.
/* comentario sin cerrar
@
```

8. Ejemplos de Uso

8.1 Ejemplo de uso correcto

Código de entrada:

```
int main() {
    return 0;
}
```

Al presionar el botón Compilar, el sistema genera una salida similar a:

```
Linea: 1 Lexema: int Token: 17 Tipo: PalabraReservada
Linea: 1 Lexema: main Token: 300 Tipo: Identificador
Linea: 1 Lexema: ( Token: 75 Tipo: Simbolo
Linea: 1 Lexema: ) Token: 76 Tipo: Simbolo
Linea: 1 Lexema: { Token: 77 Tipo: Simbolo
Linea: 2 Lexema: return Token: 20 Tipo: PalabraReservada
Linea: 2 Lexema: 0 Token: 301 Tipo: NumeroEntero
Linea: 2 Lexema: ; Token: 92 Tipo: Simbolo
Linea: 3 Lexema: } Token: 78 Tipo: Simbolo
```

Total de tokens: 9

Errores léxicos: 0

8.2 Ejemplo con números enteros y reales

Código de entrada:

```
float precio = 10.50;
double pi = 3.1416;
int edad = 25;
```

Resultado esperado:

```
float    -> PalabraReservada
precio   -> Identificador
10.50    -> NumeroReal
double   -> PalabraReservada
pi        -> Identificador
3.1416   -> NumeroReal
int       -> PalabraReservada
edad      -> Identificador
25        -> NumeroEntero
```

8.3 Ejemplo con comentarios

Código de entrada:

```
int x = 10; // variable inicial

/* Este es
   un comentario */
int y = 5;
```

El sistema reconocerá:

```
// variable inicial    -> ComentarioLinea
/* Este es ... */      -> ComentarioBloque
```

8.4 Ejemplo con errores léxicos

Código de entrada:

```
float x = 10.5.3;
int y = 8.;
/* comentario sin cerrar
```

El sistema mostrará errores como:

```
Linea: 1  Lexema: 10.5.3          Token: -1  Tipo: ErrorLexico
Linea: 2  Lexema: 8.              Token: -1  Tipo: ErrorLexico
Linea: 3  Lexema: /* comentario sin cerrar Token: -1  Tipo: ErrorLexico
```

Errores léxicos: 3

9. Recomendaciones de Uso

- Escribir código similar a C para obtener mejores resultados.
- Guardar el archivo antes de salir si se realizaron cambios.
- Revisar la consola de salida después de presionar Compilar.
- Verificar la cantidad de errores léxicos al final del análisis.
- Utilizar el botón Ayuda para consultar información adicional del proyecto.

10. Conclusión

El compilador MicroC permite realizar el análisis léxico de código fuente de manera visual y ordenada. Su función principal es reconocer lexemas, clasificarlos como tokens y mostrar información detallada sobre cada uno.

Con las Fases I y II implementadas, el sistema reconoce palabras reservadas, identificadores, números enteros, números reales, comentarios, operadores y errores léxicos, dejando preparada la base para futuras fases del compilador.